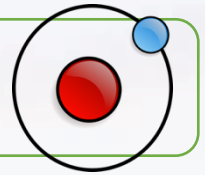
UNIVERSITY OF CAMBRIDGE

Lecture 15
Signal Emulation for Astrophysics and Cosmology

Lecturer: **Dr Harry Bevins** (htjb2)

An SKA observation

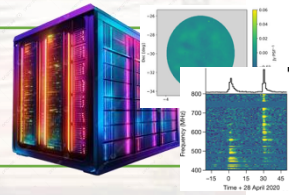
L1:
Physics



L 2,4,5:
Instrument

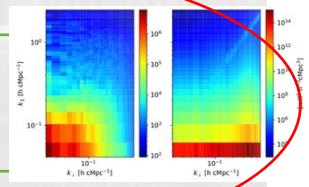
L6:
Calibration

L7-10:
Science data



L11-12:
Data handling

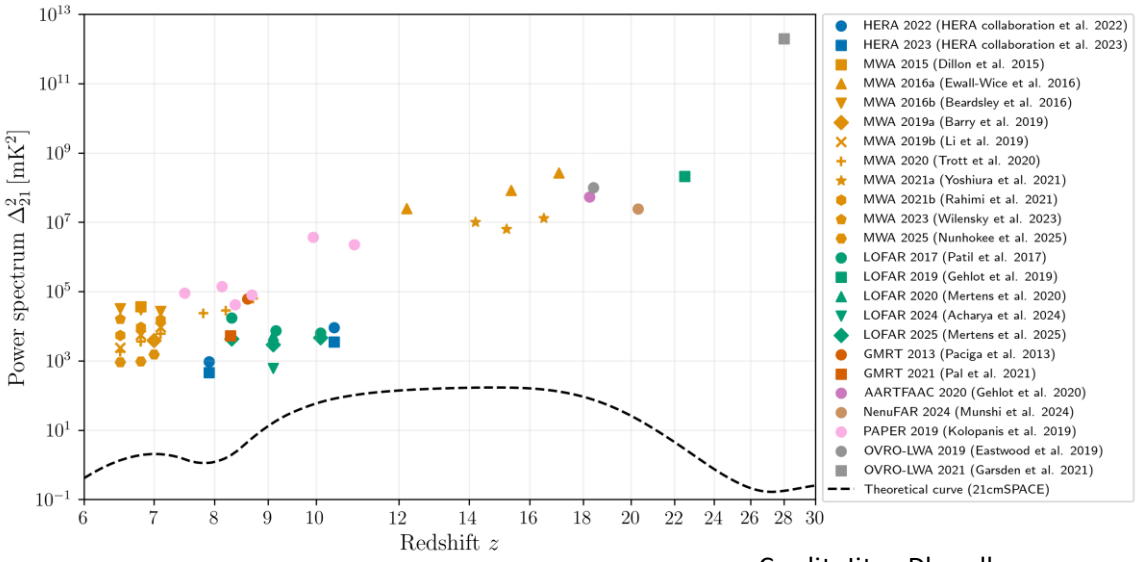
L13-16:
Science



Lectures 13-16: Science extraction

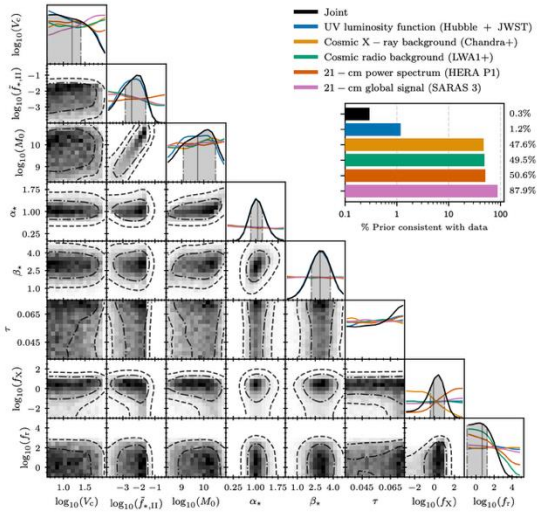
Example:

The Epoch of Re-ionization and the 21-cm line



Credit: Jiten Dhandha

From **science data products**



Dhandha et al. 2025

To **science**

UNIVERSITY OF CAMBRIDGE

Overview

- Why we need signal emulators?
- Gaussian Processes
- Neural Networks

Slides available at https://github.com/htjb/Talks (and Moodle!)

Example codes on github too!

UNIVERSITY OF CAMBRIDGE

Why we need signal emulators?

UNIVERSITY OF CAMBRIDGE

Recap:Bayes theorem

Posterior: P(θ)

Evidence: Z

Likelihood: $L(\theta)$

Prior: $\pi(\theta)$

$$\underbrace{P(\theta|D)P(D)}_{\text{Outputs}} = \underbrace{P(D|\theta)P(\theta)}_{\text{Inputs}}$$

For Nested Sampling and other MCMC algorithms

UNIVERSITY OF CAMBRIDGE

The likelihood function

- The likelihood function gives the probability of the *data given the model and parameters* $L(\theta) = P(D|\theta, M)$

- Defined as an input to Nested Sampling and other MCMC algorithms

- A lot of research goes into the form of the likelihood which varies for different analysis problems (e.g. Scheutwinkel et al. [2204.04491], Lovick et al. [2312.02075])

UNIVERSITY OF CAMBRIDGE

An example likelihood

$$\log_e L(\theta) = \sum_i -\frac{1}{2}\log_e |2\pi\Sigma| - \frac{1}{2}\left(D_i - M_i(\theta)\right)^T \Sigma^{-1}(D_i - M_i(\theta))$$

- Work in log space to make things more computationally stable
- The functional form of the likelihood defines the distribution of the random component of your data
- Here we are assuming the noise (or random component) is Gaussian and allowing for a non-diagonal covariance

UNIVERSITY OF CAMBRIDGE

The issue

$$\log_e L(\theta) = \sum_i -\frac{1}{2}\log_e|2\pi\Sigma| - \frac{1}{2}\left(D_i - M_i(\theta)\right)^T \Sigma^{-1}(D_i - M_i(\theta))$$

- In an MCMC run the likelihood function is evaluated for $\sim 10^5 - 10^6$ different θs
- This means we potentially need to evaluate the model $M(\theta)$ millions of times
- But the models are often computationally expensive taking minutes to hours to days per realization

UNIVERSITY OF CAMBRIDGE

The issue

$$\log_e L(\theta) = \sum_i -\frac{1}{2}\log_e|2\pi\Sigma| - \frac{1}{2}\left(D_i - M_i(\theta)\right)^T \Sigma^{-1}(D_i - M_i(\theta))$$

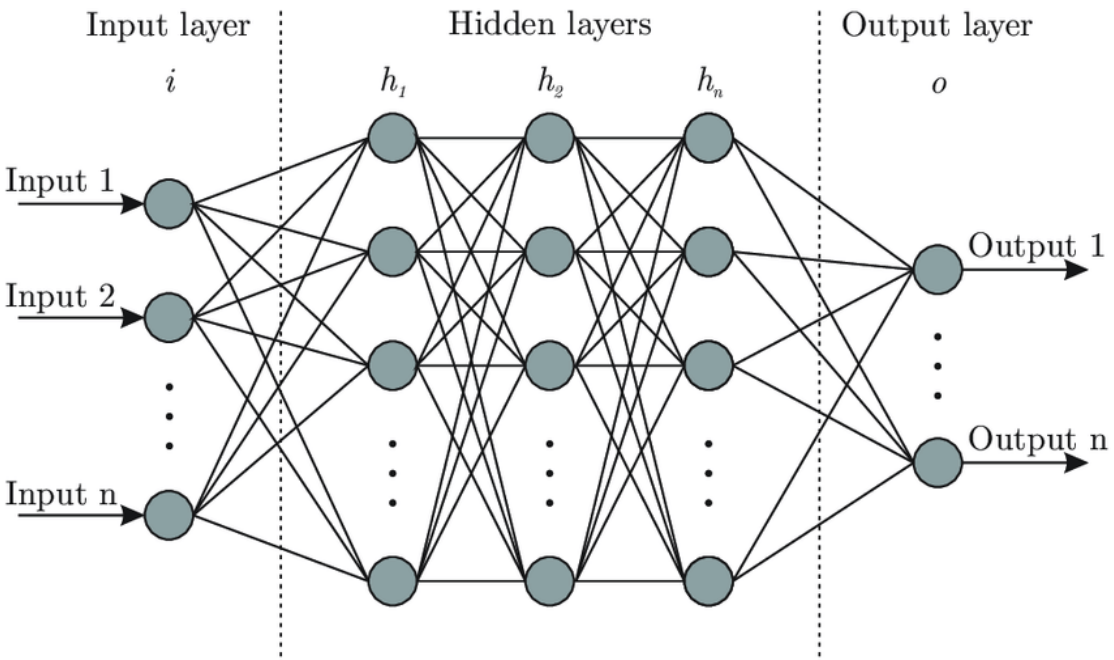
- How do we get around this?
- Look towards emulators which can typically approximate $M(\theta)$ in milliseconds
- Go from millions of hours to hours or even minutes per MCMC chain

UNIVERSITY OF CAMBRIDGE

What actually is an emulator?

- Some approximation of a complex astrophysical model (think N-body, hydrodynamical or semi-numerical simulations)

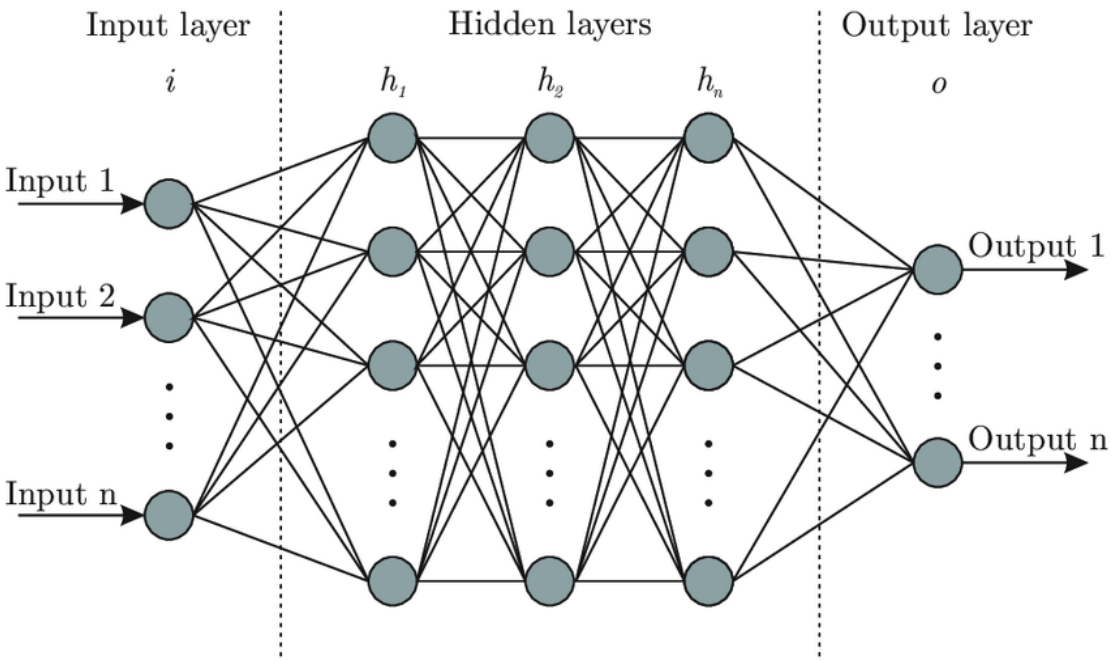
- Built with machine learning tools like ANNs, CNNs and GPs



UNIVERSITY OF CAMBRIDGE

Emulators as an approximation

- In $g(\theta) \approx f_{\phi}(\theta)$, ϕ are the parameters of our emulator
- ϕ has to be optimized so that the approximation is accurate as possible
- Can say $g(\theta) = f_{\phi}(\theta) + \epsilon_{\phi}(\theta)$ where we are attempting to minimize the error $\epsilon_{\phi}(\theta)$



UNIVERSITY OF CAMBRIDGE

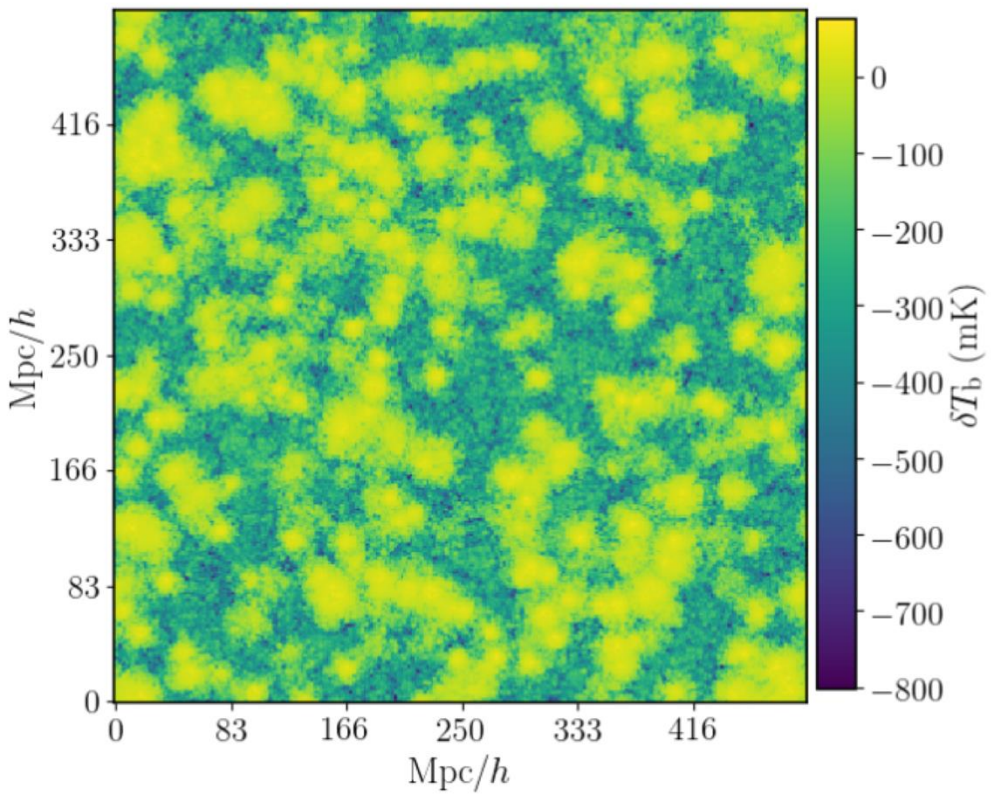
21-cm Cosmology

UNIVERSITY OF CAMBRIDGE

21-cm Cosmology

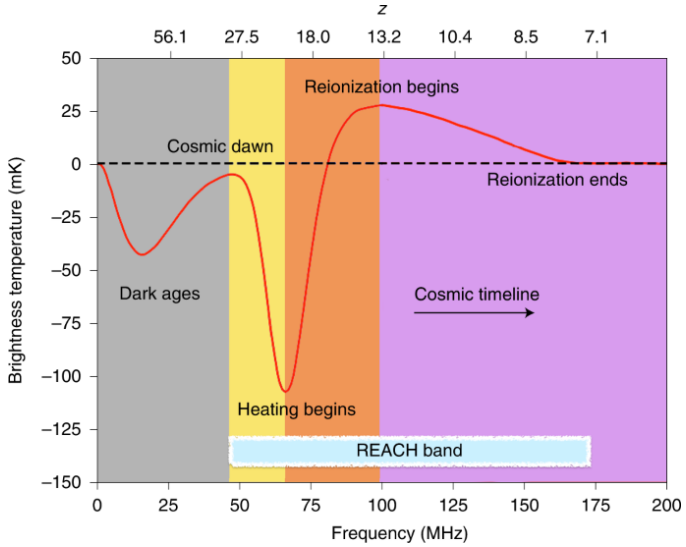
- Tracing the evolution of the universe through observations of the 21-cm signal from neutral hydrogen

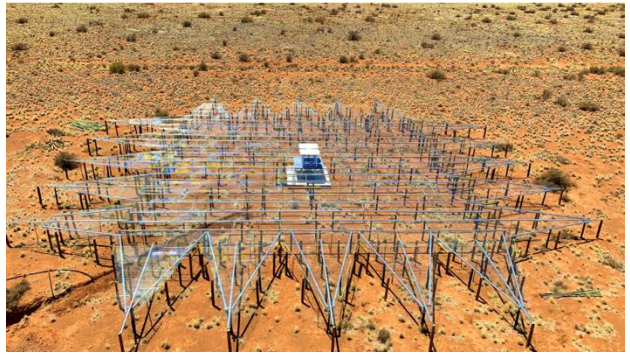
- A detection of the signal will help us understand
  - When the first stars formed ad how bright they were
  - The nature of dark matter
  - The abundance and brightness of X-ray emitting objects
  - When the universe transformed from neutral to ionized

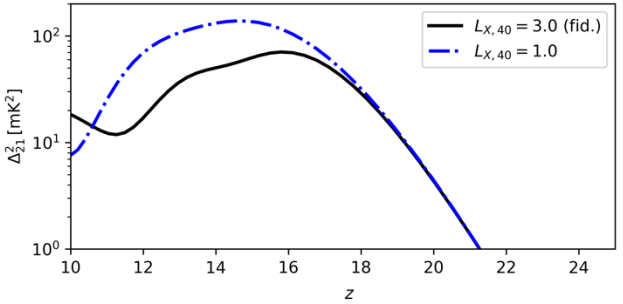


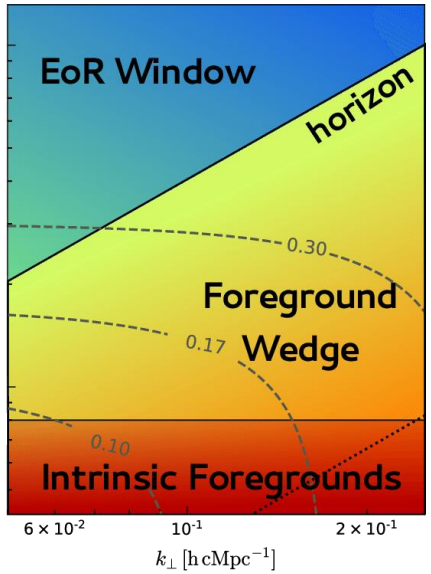
UNIVERSITY OF CAMBRIDGE

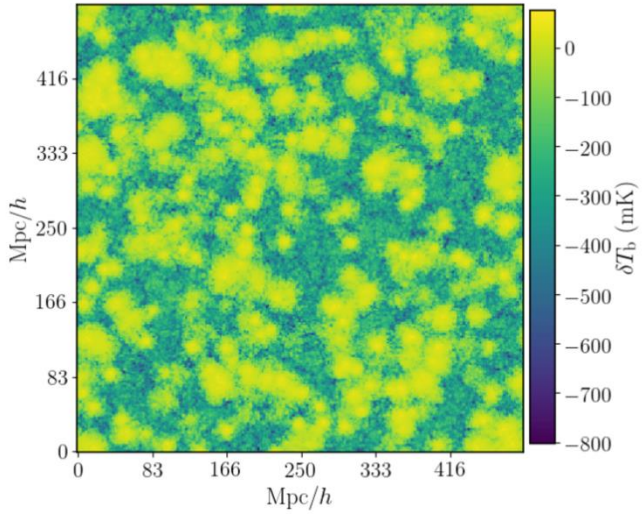
21-cm Cosmology







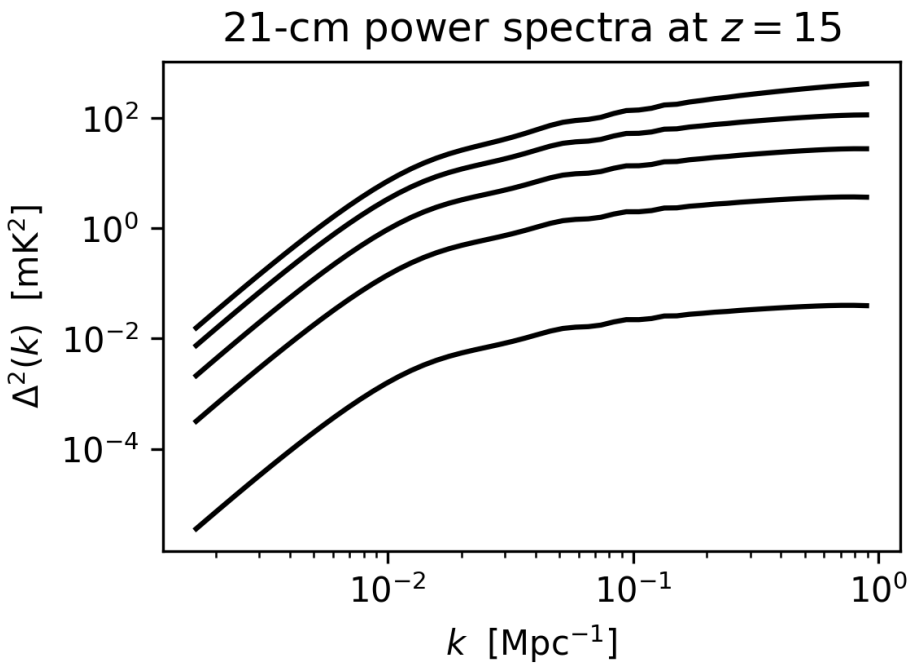




UNIVERSITY OF CAMBRIDGE

21-cm Cosmology

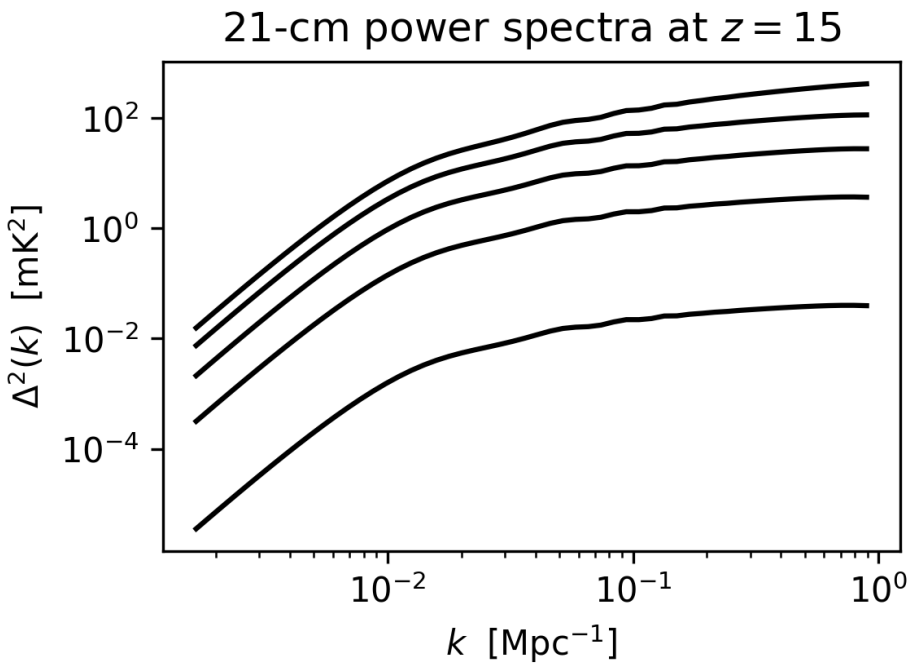
- Complex dependence between $\Delta^2(k)$ and the parameters of our models θ
- We use simulations to model and study how the signal varies over space and time for given θ
- Here we are using Zeus21 which is a fast approximate model of the 21-cm signal over time
- Zeus21 takes < second to run per realization however more accurate semi-numerical codes takes $\approx$ 3hrs



UNIVERSITY OF CAMBRIDGE

21-cm Cosmology

- The zeus21 model has many different parameters
- There are 10,000 example signals to use to build our emulators
- For the examples we are fixing a lot of them and just looking at the effects of
  - $L_{X,40}$ - X-ray luminosity
  - ϵ – star formation efficiency
  - $f_{esc,10}$ - escape fraction of ionized photons
  - h_0 - Hubble's constant



UNIVERSITY OF CAMBRIDGE

Gaussian Process

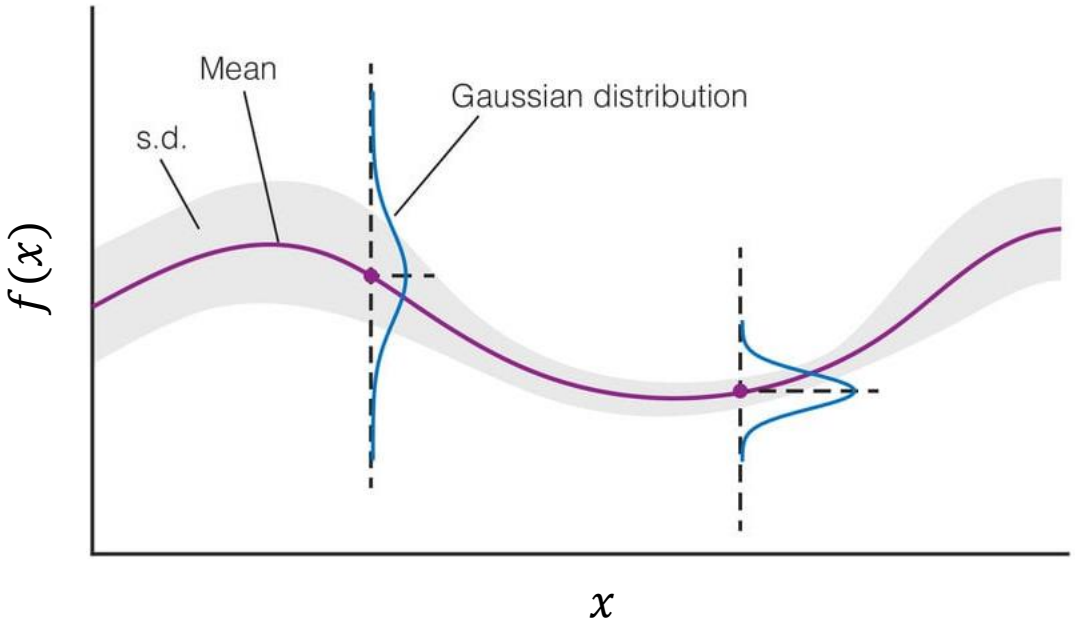
UNIVERSITY OF CAMBRIDGE

What is a Gaussian Process?

- The idea is to parameterize the distribution of all functions with a Gaussian

$$\hat{f}(x) \sim N\big(\mu(x), C(x, x')\big)$$

- $\mu(x)$ is the mean function
- $C(x, x')$ is the covariance
- Typically see this in regression problems as a non-parameteric or model independent fitting technique



UNIVERSITY OF CAMBRIDGE

How do we use it to emulate a model?

- We have a set of simulations $f(x, \theta)$

- We decompose our data such that we have

$$f(x, \theta) = \sum_i^N w_i(\theta) g(x)$$

where w_i are weights and $g(x)$ is independent of the parameters

- We can then define N GPs

$$w_i(\theta) \sim N\big(\mu_i(\theta), C_i(\theta, \theta')\big)$$

where μ_i and C_i can be parameterized by ϕ and ϕ is optimized with data examples

UNIVERSITY OF CAMBRIDGE

Dimensionality Reduction

- This idea

$$f(x,\theta) = \sum_{i}^{N} w_i(\theta) g(x)$$

can be very powerful

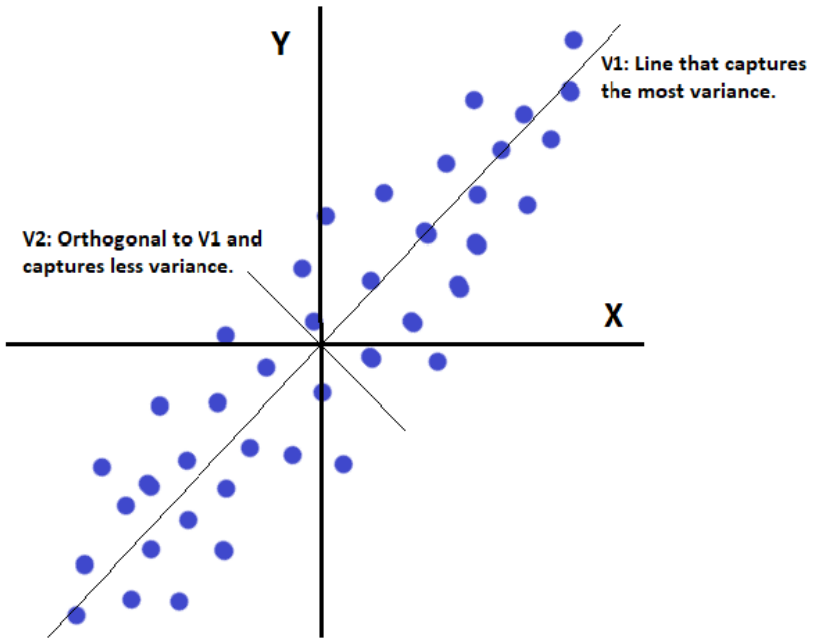
- If we make a good choice for $g(x)$ then the dimensionality of our problem reduces from N_x to N_w

- These techniques can lead to information loss, but it can be managed

UNIVERSITY OF CAMBRIDGE

Principle Component Analysis

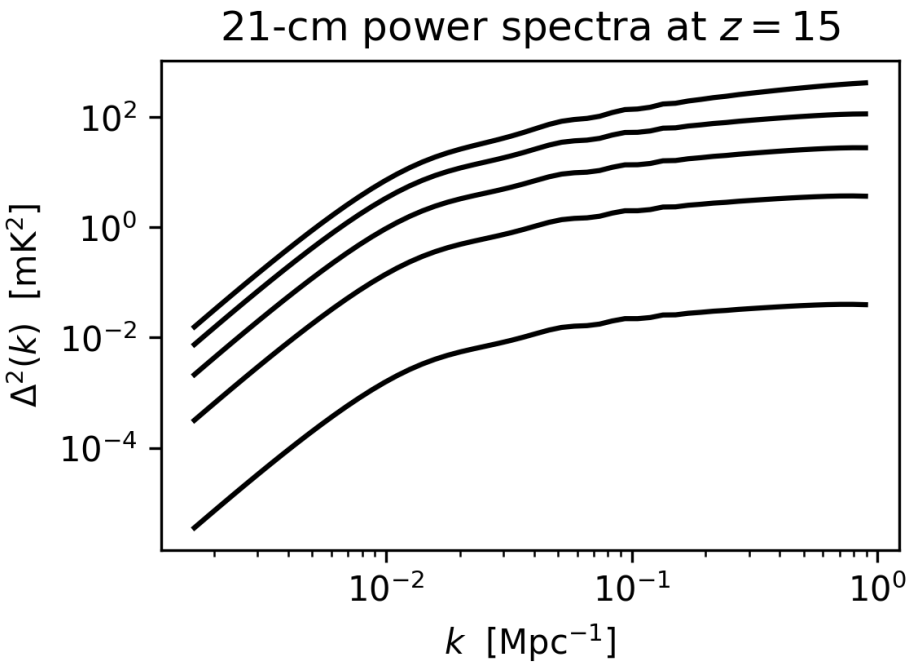
- Decompose training data into eigenvectors (e.g. $g(x)$) and order based on eigenvalues
- Eigenvectors with largest eigenvalues describe the directions in the training data with the biggest variance
- Discard eigenvectors that correspond to low variance directions
- Reconstruct signals with weighted sum (e.g. $w_i(\theta)$) of most important eigenvectors where weights vary for each signal

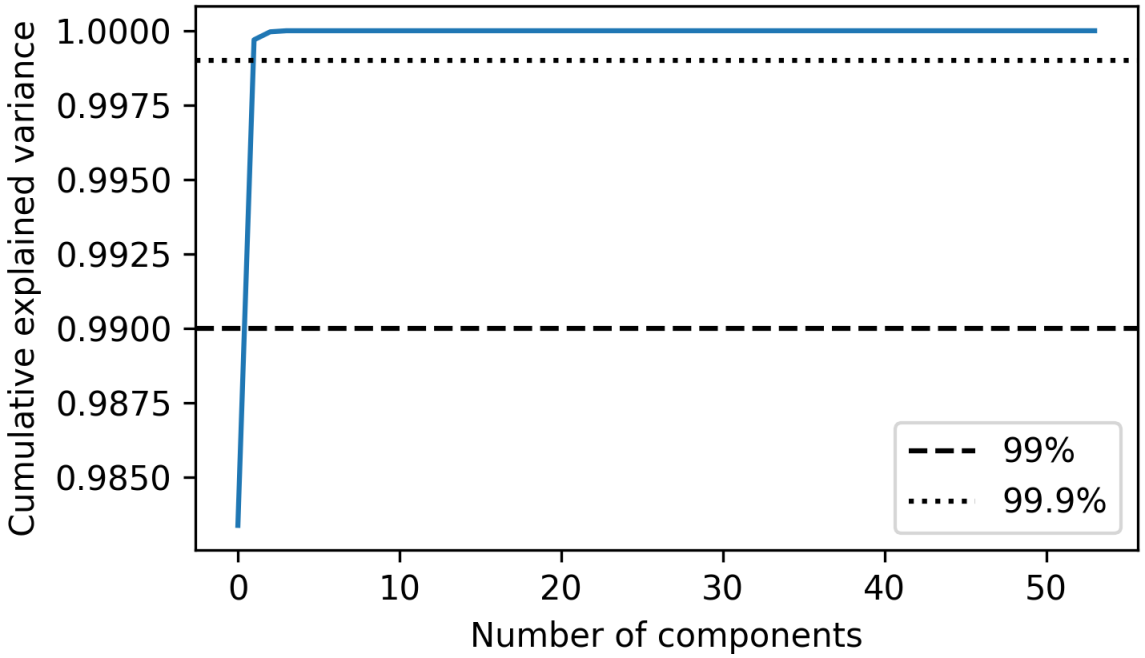


UNIVERSITY OF CAMBRIDGE

Principle Component Analysis

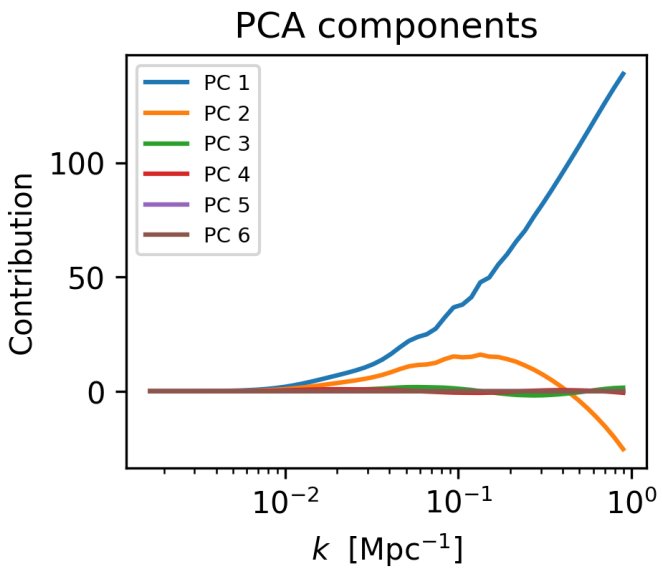
Cumulative explained variance for m eigenvectors out of N → $$CEV = \frac{\sum_{k=0}^{m} \lambda_k}{\sum_{k=0}^{N} \lambda_k}$$

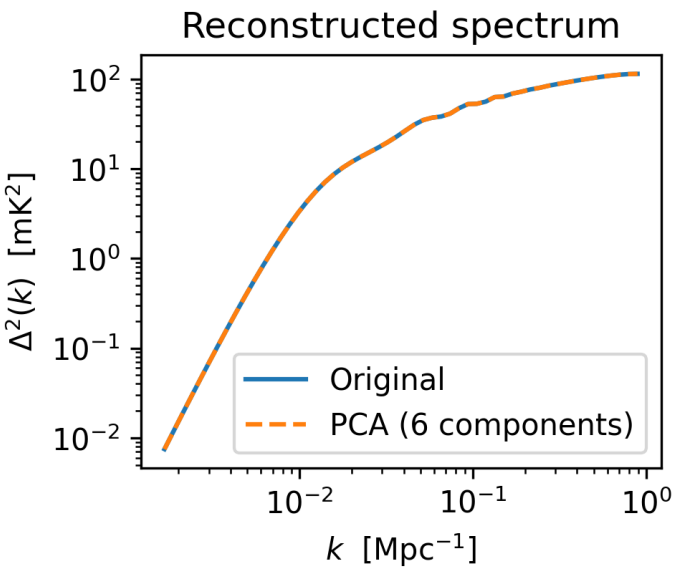


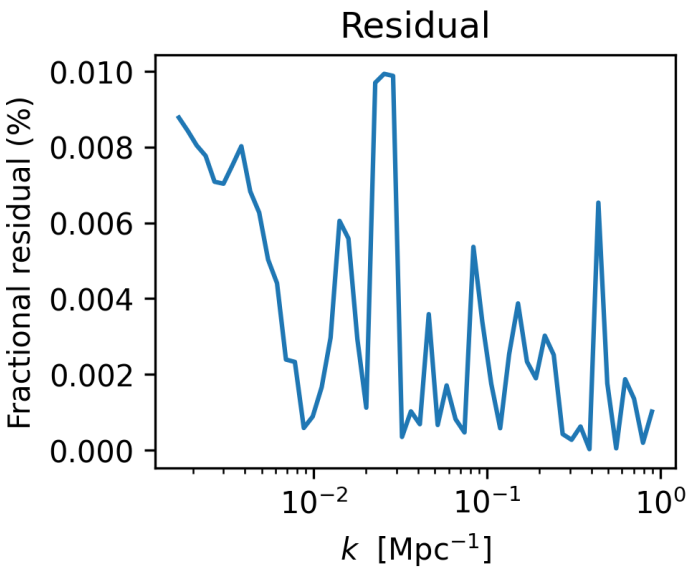


UNIVERSITY OF CAMBRIDGE

Principle Component Analysis







$$g(x) = g(k)$$

$$f(x,\theta) = \sum_i^N w_i(\theta)\, g(k)$$

$$FR = 100 \times \frac{|f(x,\theta) - \Delta^2(k)|}{|\Delta^2(k)|}$$

UNIVERSITY OF CAMBRIDGE

GP Regression with scikit learn

- We perform a PCA decomposition on our training data and find we need 3 components to explain our data set
- We therefore want 3 GPs to predict the weights as a function of the model parameters
- Define our kernel as

$$k(\theta, \theta') = \sigma^2 \exp\left(-\frac{1}{2}\sum_{d=1}^{4}\frac{(\theta_d - \theta'_d)^2}{l_d^2}\right)$$

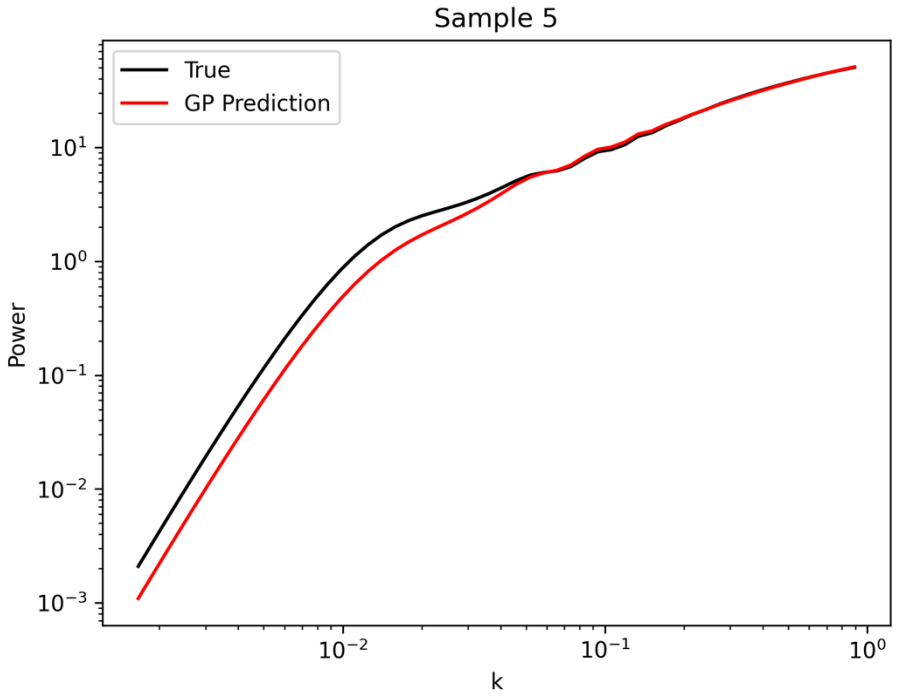
- Goal is to maximise the likelihood of the training examples under this model

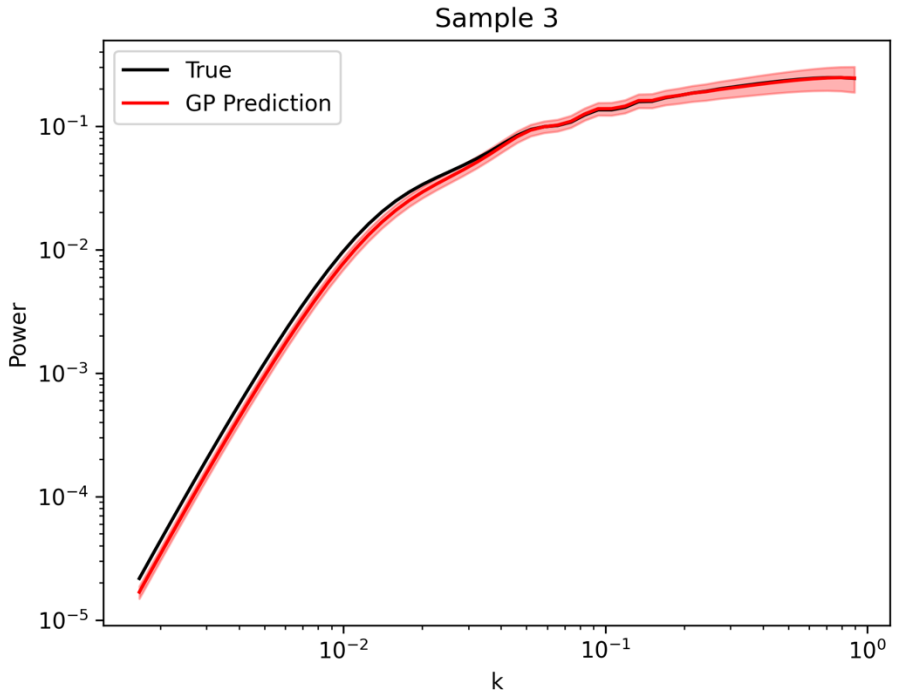
```python
kernel = ConstantKernel(1.0) * RBF(length_scale=[1.0]*4)

gps = []
for i in range(n_components):
    gp = GPR(
        kernel=kernel,
        alpha=1e-6,
        n_restarts_optimizer=5,
        random_state=i,
    )
    gp.fit(train_X, train_pca[:, i])
    gps.append(gp)
    print(f"GP {i+1} trained | log-marginal-likelihood =" +
          f"{gp.log_marginal_likelihood_value_:.2f}")
```

UNIVERSITY OF CAMBRIDGE

Assessing performance

$$\theta \rightarrow \text{Trained GPs} \rightarrow w_i(\theta) \text{ for } i = 0 \ldots 3 \rightarrow f(\theta, x) = \sum_i^N w_i(\theta) g(x)$$



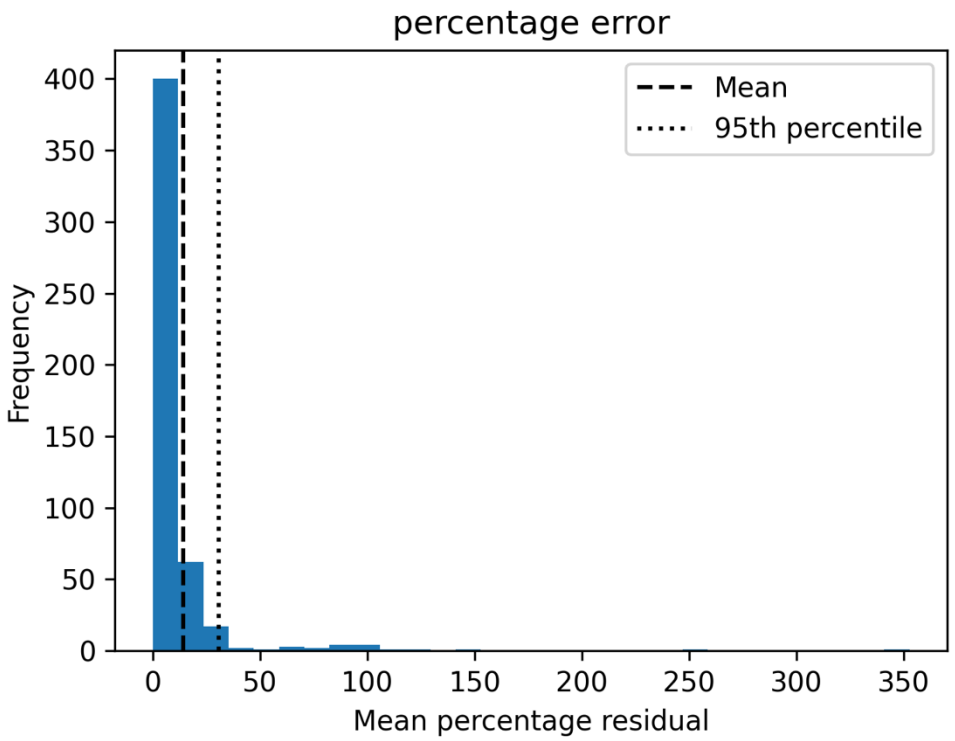


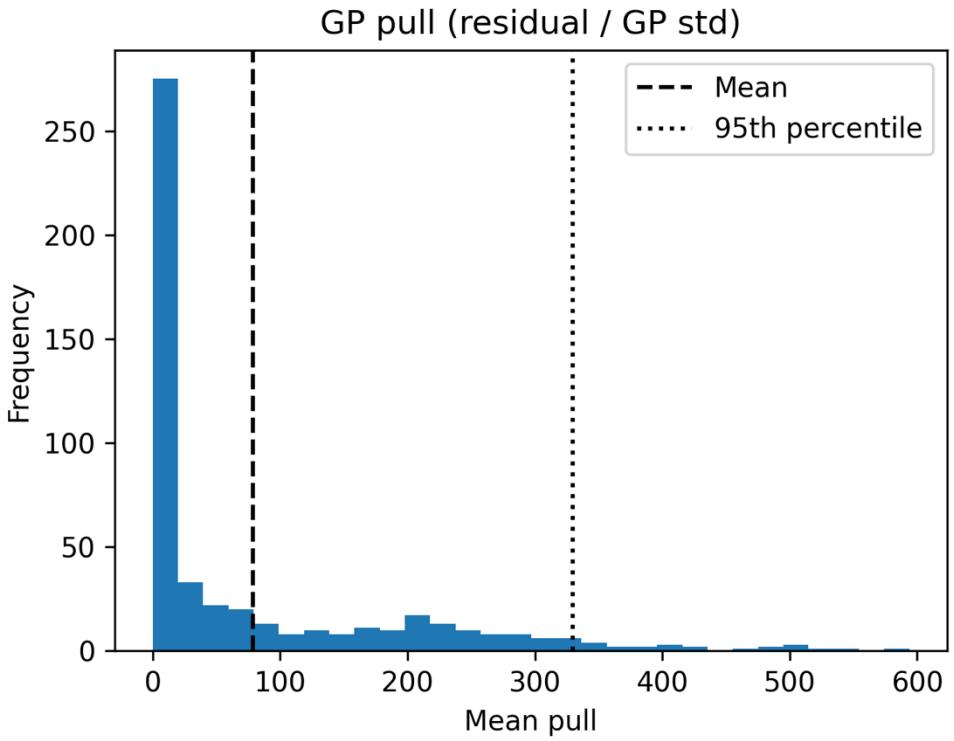
UNIVERSITY OF CAMBRIDGE

Assessing performance

$$\epsilon_i = 100\,\frac{1}{N_k}\sum_k \frac{|\Delta^2_{true}(k) - \Delta^2_{pred}(k)|}{|\Delta^2_{true}(k)|}$$

$$\epsilon_i = \frac{1}{N_k}\sum_k \frac{|\Delta^2_{true}(k) - \Delta^2_{pred}(k)|}{|\sigma_\Delta(k)|}$$



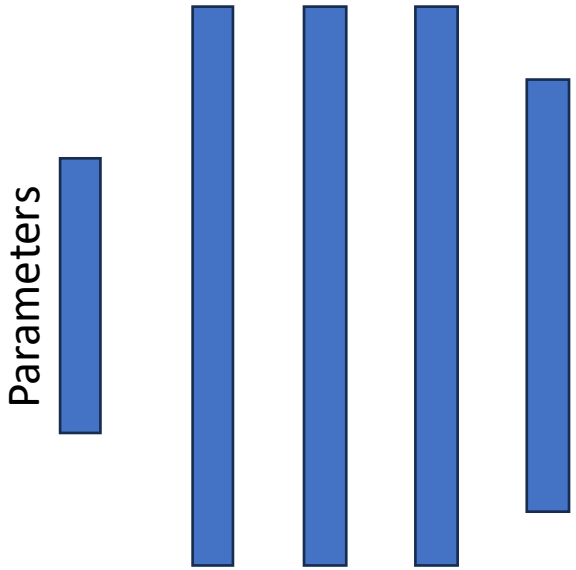


UNIVERSITY OF CAMBRIDGE

Neural Networks

UNIVERSITY OF CAMBRIDGE

Principle Component Analysis



Signal Specific PCA coefficients $w_i(\theta)$ $\times$ Important eigenvectors $g(x)$ $\longrightarrow$ Sum Terms

$$f(x,\theta) = \sum_i^N w_i(\theta) g(x)$$

NNs are a bit more flexible than GPs so we have to think more carefully about some of our design choices

UNIVERSITY OF CAMBRIDGE

Loss Function

- As with the GPR, we provide the network with training data $\{\Delta^2, \theta\}$ but there is a bit more freedom in our objective function
- Recall we are trying to find the best set of parameters ϕ to minimise the error in our approximation ϵ_ϕ
- Two common choices are the MAE and MSE

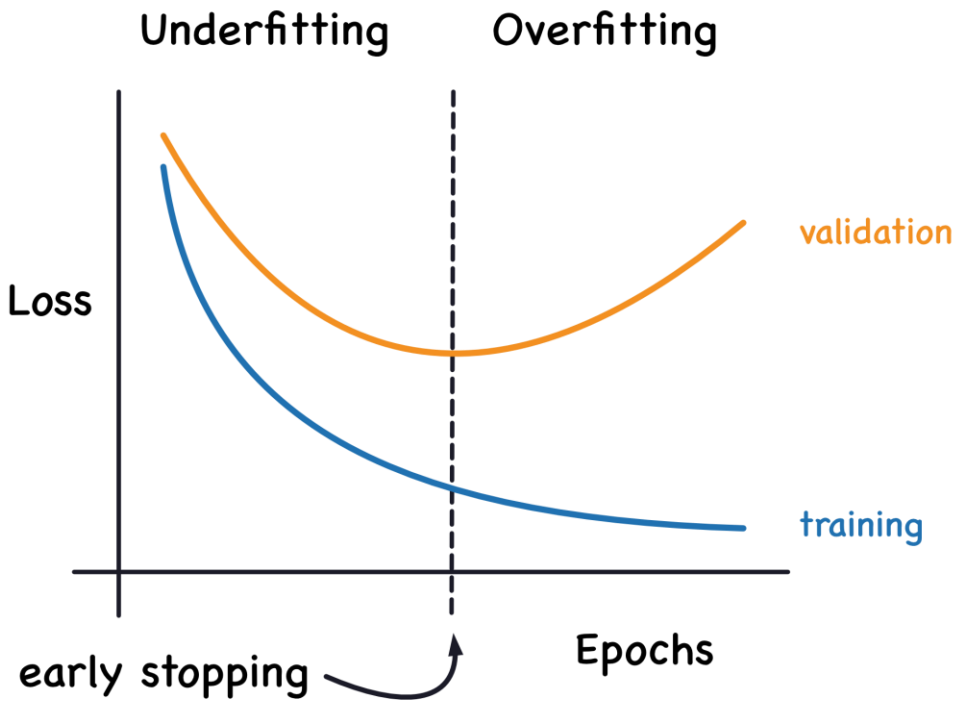
$$L = \frac{1}{N}\sum_i \left|\epsilon_\phi(\theta)\right| \qquad L = \frac{1}{N}\sum_i \left(\epsilon_\phi\right)^2$$

- NNs are typically optimized with some form of Stochastic Gradient descent

UNIVERSITY OF CAMBRIDGE

Training, test and validation data

- We reserve a proportion of our data for validating the performance of the emulator during training (early stopping)



UNIVERSITY OF CAMBRIDGE

Building the neural network

Many frameworks for building NNs in Python including Tensorflow, JAX and PyTorch…

```
class Emulator(nn.Module):
    def __init__(self, input_dim, output_dim, hidden_dim=16, num_layers=3):
        super(Emulator, self).__init__()
        input = nn.Linear(input_dim, hidden_dim)
        output = nn.Linear(hidden_dim, output_dim)

        self.layers = nn.ModuleList()
        self.layers.append(input)
        for i in range(num_layers):
            self.layers.append(nn.Linear(hidden_dim, hidden_dim))
        self.layers.append(output)

    def forward(self, x):
        x = torch.relu(self.layers[0](x))
        for layer in self.layers[1:-1]:
            x = torch.relu(layer(x))
        x = self.layers[-1](x)
        return x
```

UNIVERSITY OF CAMBRIDGE

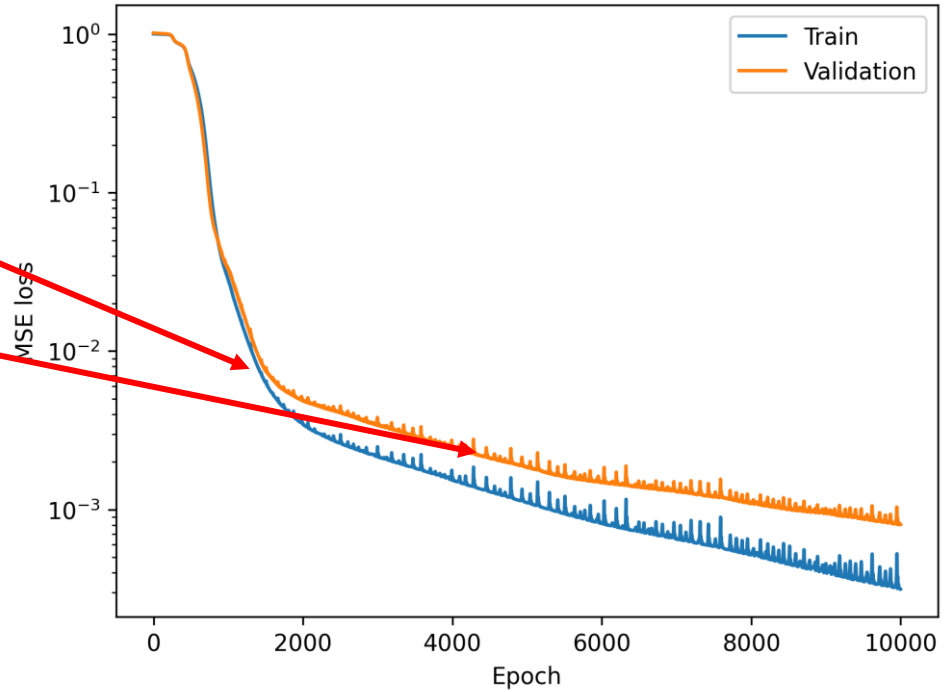
Training

```python
n_epochs = 10000
patience = 100
counter  = 0
best_val_loss    = float('inf')
best_model_state = model.state_dict()
train_losses, val_losses = [], []

pbar = tqdm(range(n_epochs))
for epoch in pbar:
    model.train()
    optimizer.zero_grad()
    loss = criterion(model(train_X_tensor), train_pca_tensor)
    loss.backward()
    optimizer.step()

    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(val_X_tensor), val_pca_tensor)

    train_losses.append(loss.item())
    val_losses.append(val_loss.item())

    if val_loss < best_val_loss:
        best_val_loss    = val_loss
        best_model_state = model.state_dict()
        counter = 0
    else:
        counter += 1
        if counter >= patience:
            print(f'Early stopping at epoch {epoch+1}')
            break

    pbar.set_description(
        f'Loss: {loss.item():.4f}  Val: {val_loss.item():.4f}'
    )

model.load_state_dict(best_model_state)
```

Training Step

Validation Step

Early Stopping



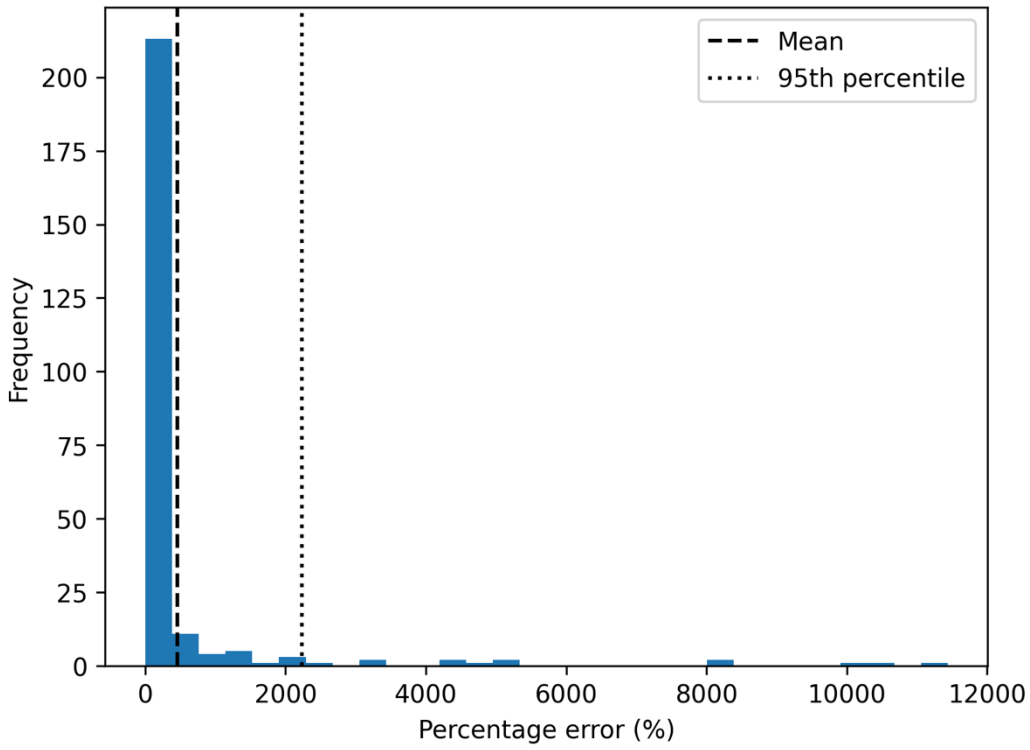
UNIVERSITY OF CAMBRIDGE

Assessing the accuracy

$$\epsilon_i = 100 \frac{1}{N_k} \sum_k \frac{|\Delta^2_{true}(k) - \Delta^2_{pred}(k)|}{|\Delta^2_{true}(k)|}$$

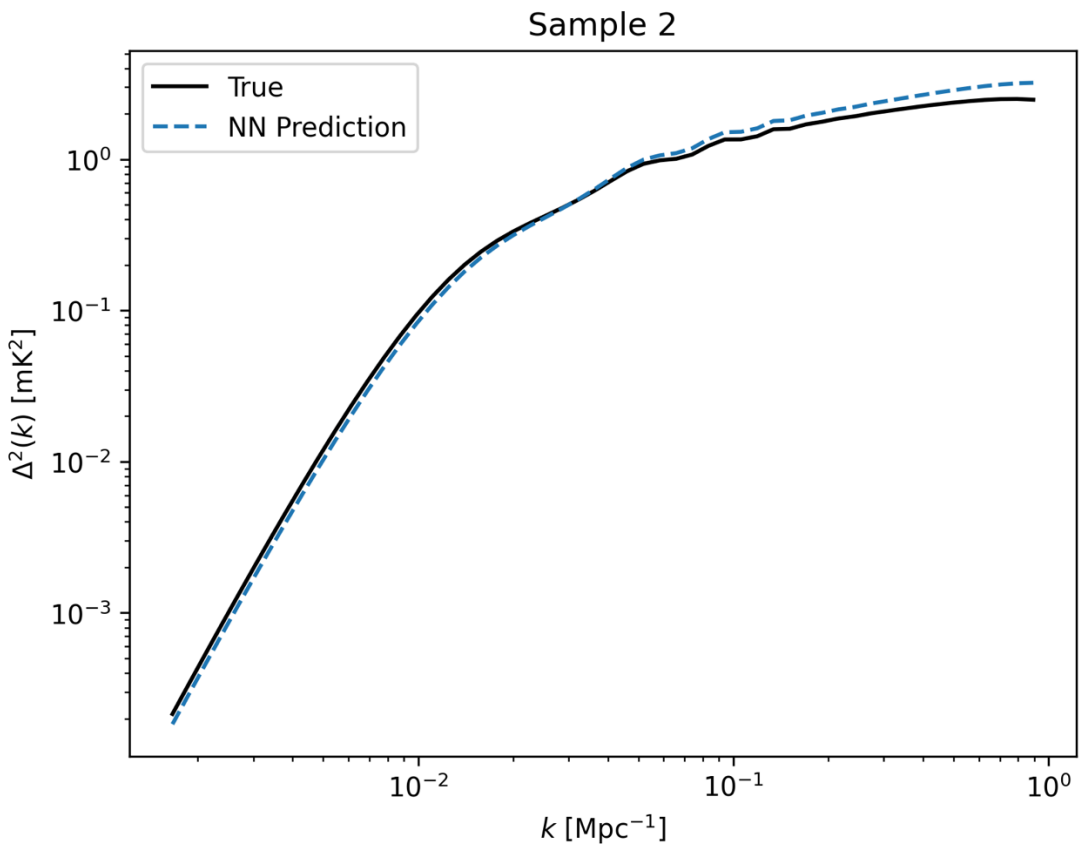
- We assess on the whole test data set and take the average and 95th percentile values

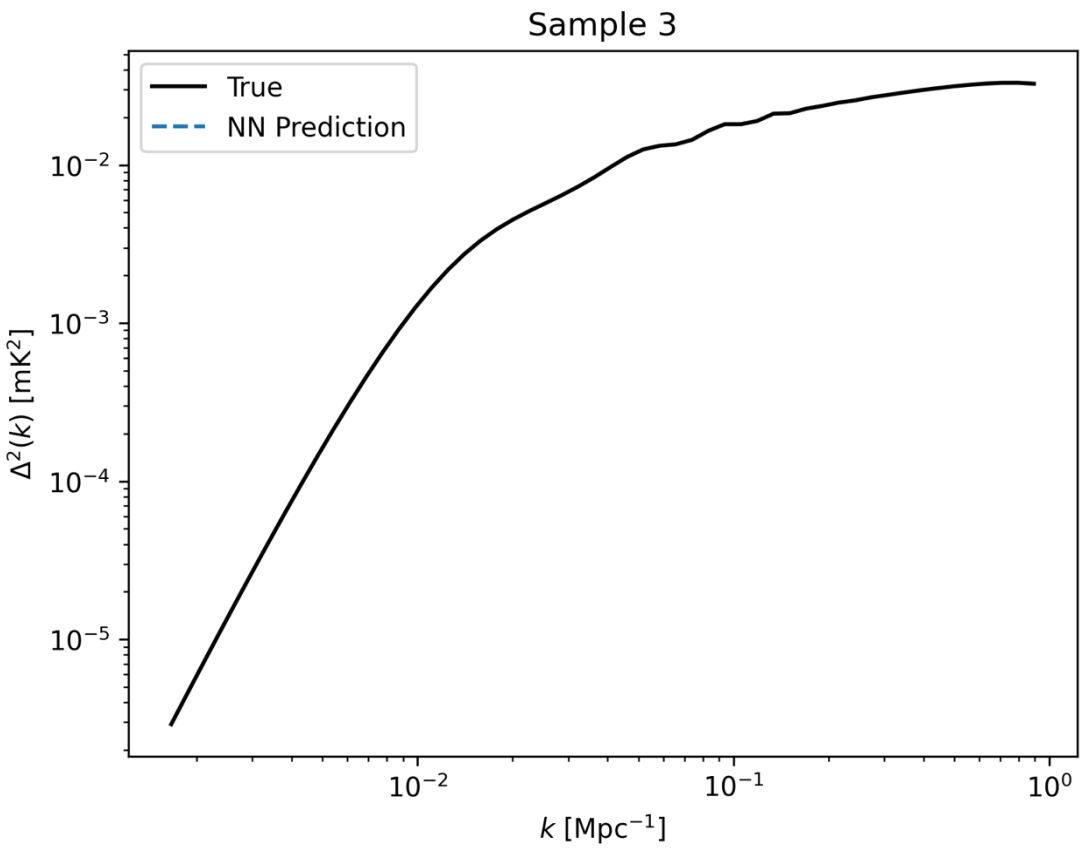
- In this case $\bar{\epsilon} \approx 456.693$ % and $\epsilon_{95} \approx 2230.327$ %



UNIVERSITY OF CAMBRIDGE

Assessing the accuracy



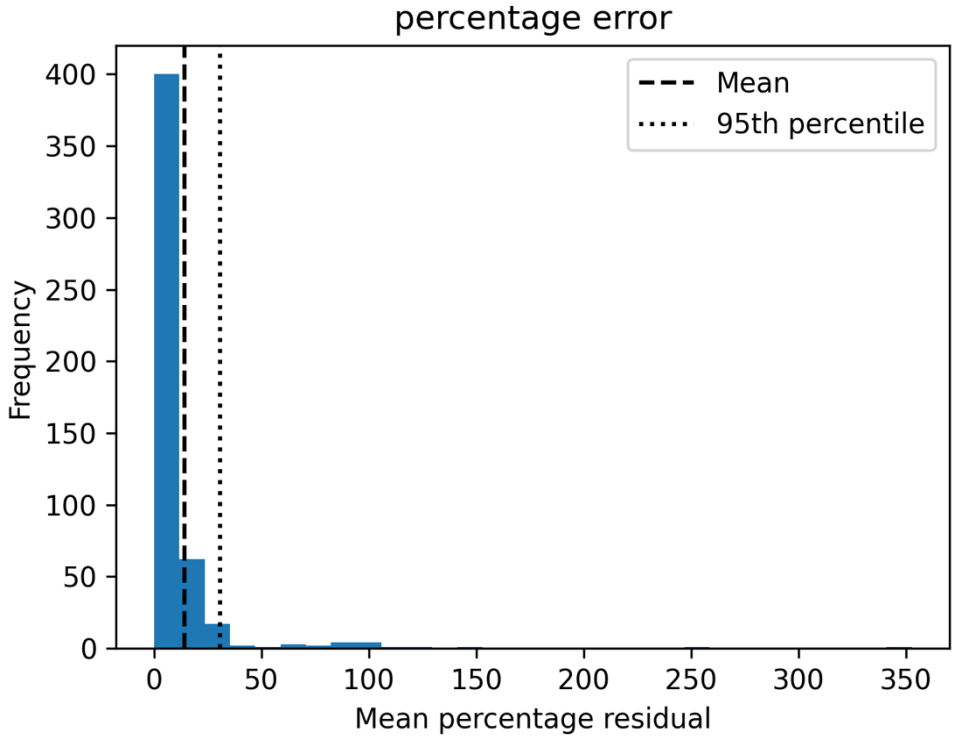


UNIVERSITY OF CAMBRIDGE

Comparing with the GP

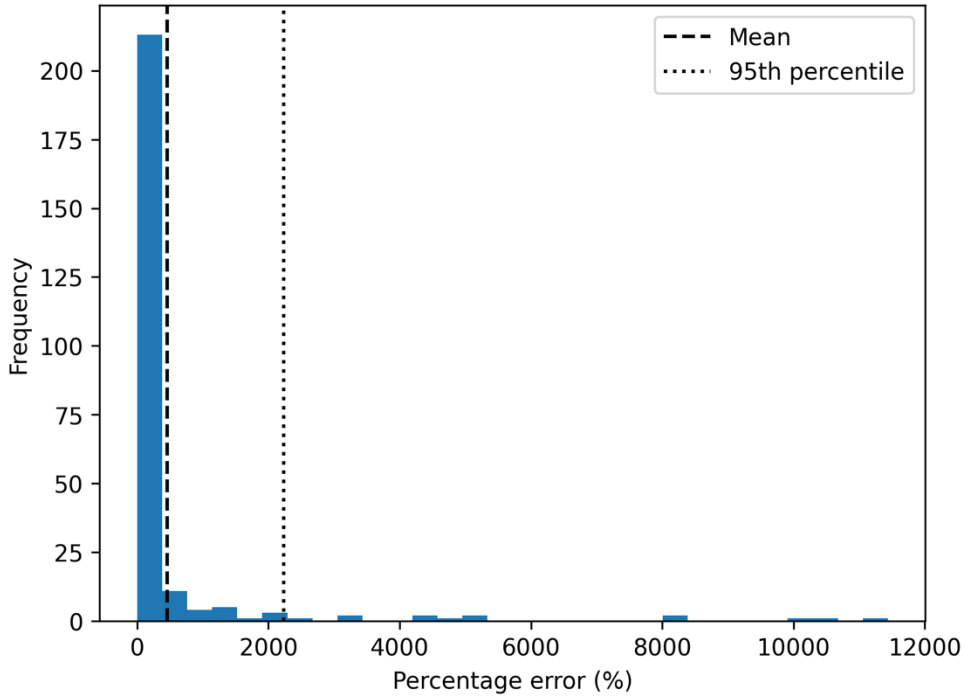
GP:

- Mean percentage error: 14.12 %
- 95th percentile: 30.82%



NN:

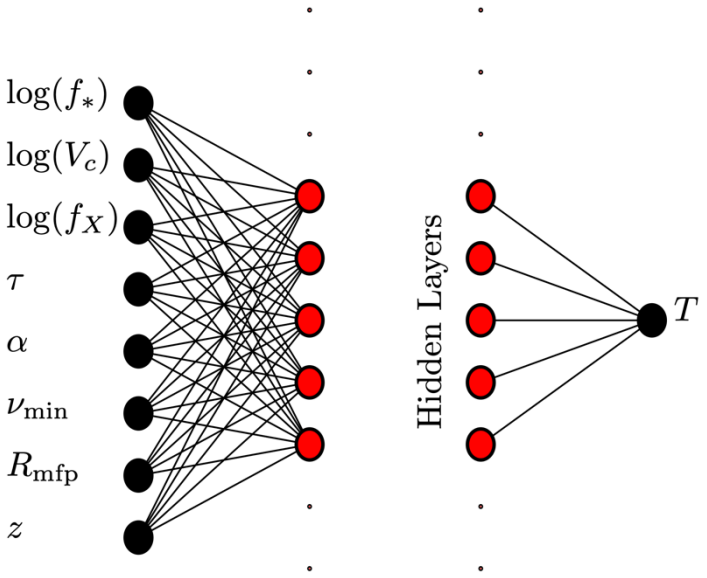
- Mean error: 456.693 %
- 95th percentile: 2230.327 %

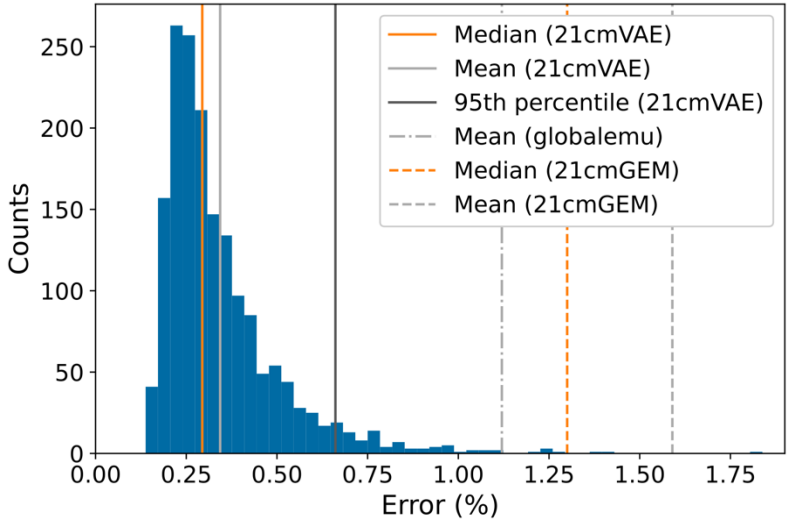


UNIVERSITY OF CAMBRIDGE

Current state of the art

- globalemu [2104.04336] uses small MLPs
- 21cmVAE [2107.05581] uses Variational Autoencoders
- 21cmLSTM [2410.07619] uses long and short term memory networks
- 21cmKAN [2508.11752] uses Kolmogrov-Arnold networks





UNIVERSITY OF CAMBRIDGE

Summary

UNIVERSITY OF CAMBRIDGE

Emulators allow us to do inference

- Emulators are an efficient way to approximate complex simulations
- They take of order milliseconds to evaluate compared to minutes, hours or even days per realization making inference possible
- Sensible choices about architectures, use of dimensionality reduction techniques, loss functions etc can make a big difference to the run time and accuracy of emulators

UNIVERSITY OF CAMBRIDGE

Many different ways to do this

- Lots of different ways to build emulators
  - Here we have looked at Dense Neural Networks and Gaussian Processes
  - But we can also do this with long and short term memory networks, Kolmogrov-Arnold networks, variational autoencoders, convolutional neural networks…

Slides available at https://github.com/htjb/Talks (and Moodle!)
Example codes on github too!

UNIVERSITY OF CAMBRIDGE

Next Lecture: Simulation Based Inference

- Why not just emulate the whole likelihood?